



Centro Regional Universitario Córdoba IUA  
**Ing. en Informática**

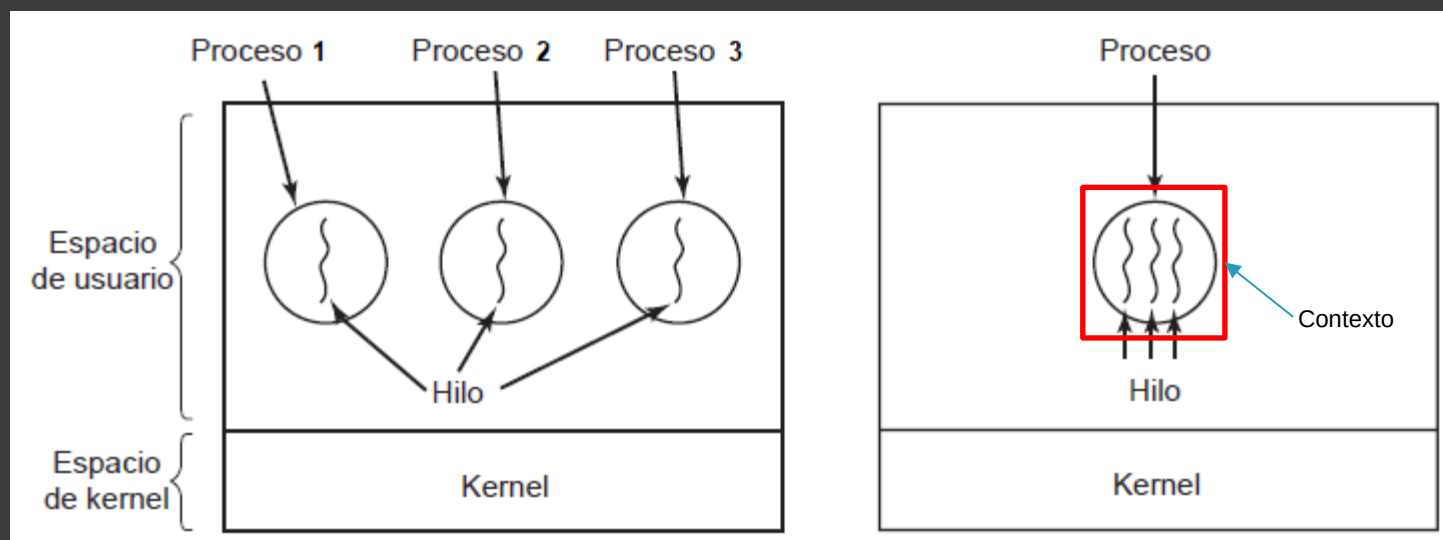
# **SISTEMAS EN TIEMPO REAL**

Mag. Ing. José Ducloux  
[jducloux@iua.edu.ar](mailto:jducloux@iua.edu.ar)

2026

# POSIX en C

- Hilos
- Procesos
- **Sincronización de hilos**
- Sincronización de procesos



# Sincronización de hilos

- Evitar condiciones de carrera entre hilos.
- Evitar carrera de datos entre hilos.
- Establecer puntos de reunión y/o espera.
- POSIX: uso de la biblioteca **pthread** con diferentes mecanismos de control de concurrencia.

# Sincronización de hilos de POSIX

- Mecanismos de control en **pthread**
  - **Mutex** (exclusión mutua): protege el acceso a secciones críticas en recursos compartidos.
  - **Variable de condición**: espera una condición específica con notificación de eventos.
  - **Barrera**: facilita la sincronización creando puntos de encuentro.
  - **Semáforo**: permite el acceso a uno o más hilos a la vez en secciones críticas.
- Generalmente se utiliza una combinación de estos mecanismos.

# Mutex POSIX

- Permite que sólo un hilo ingrese a la sección crítica.
- Se garantiza la integridad de los datos.
- Se los conoce también como semáforos binarios: semáforos con dos estados.
- Si un hilo pretende ingresar a la sección crítica y el mutex está bloqueado, el hilo permanecerá en modo sleep hasta que se libere el mutex.
- Funciones:
  - `pthread_mutex_init()`: inicializa el mutex.
  - `pthread_mutex_lock()`: bloquea el mutex.
  - `pthread_mutex_unlock()`: desbloquea el mutex.

# Mutex POSIX

- Ejemplo 1
  - Resuelve la carrera de datos del ejemplo 5 de la presentación de hilos:
    - Carrera de datos mediante un estado compartido (variable compartida).
    - Los hilos compiten para leer/escribir datos.
    - El ejemplo usa una variable compartida del tipo entera.
    - Las salidas aceptables serían:
      - Que se imprima: 1 3
      - Que se imprima: 2 3
    - Una salida incorrecta sería: 1 2
  - Se utiliza un mutex para que sólo un hilo ingrese a la sección crítica para realizar la escritura de la variable compartida.
  - Siempre se verá 1 3 o 2 3 como salidas correctas en pantalla.

# Mutex POSIX

- Ejemplo 1
  - Se declara un mutex global.
  - Se tienen dos funciones complementarias que contienen las secciones críticas.
  - En `main()` se inicializa el mutex:
    - `pthread_mutex_init()`
  - El mutex protege las secciones críticas comprendidas entre las funciones:
    - `pthread_mutex_lock(&mtx)`
    - `pthread_mutex_unlock(&mtx)`
    - Las funciones están presentes en ambas funciones complementarias.
  - En todo momento, sólo un hilo puede estar en la sección crítica; el otro espera fuera de su sección crítica.
  - Sigue presente la condición de carrera ya que las salidas pueden ser 1 3 o 2 3.

# Spinlocks POSIX

- Similar a mutex pero utiliza espera activa (busy wait).
- Las funciones se encuentran en **pthread** con una interfaz similar a las de mutex.
- Los tipos y funciones relacionados con spinlock son los siguientes.
  - `pthread_spinlock_t`: el tipo utilizado para crear un objeto spinlock. Es similar al tipo `pthread_mutex_t`.
  - `pthread_spin_init()`: inicializa un objeto spinlock. Es similar a `pthread_mutex_init`.
  - `pthread_spin_destroy()`: similar a `pthread_mutex_destroy`.
  - `pthread_spin_lock()`: similar a `pthread_mutex_lock`.
  - `pthread_spin_unlock()`: similar a `pthread_mutex_unlock`.

# Variables de condición POSIX

- Un hilo espera el cambio de estado en una variable de condición hasta que se le notifica que despierte.
- Todas las operaciones deben estar protegidas por un mutex.
- Eliminan las condiciones de carrera entre hilos.
- Funciones:
  - `pthread_cond_init()`: inicializa la estructura de datos de la variable de condición `pthread_cond_t`.
  - `pthread_cond_destroy()`: destruye la variable de condición.
  - `pthread_cond_signal()`: notifica a un hilo el cambio en la variable de condición.
  - `pthread_cond_wait()`: uno o varios hilos esperan la notificación de cambio en la variable de condición. Se pasa también como argumento el puntero al mutex asociado.

# Variables de condición POSIX

- Ejemplo 2
  - Se utiliza como referencia el ejemplo 3 de la presentación de hilos.
    - El programa crea tres hilos casi al mismo tiempo y cada uno de ellos imprime una cadena de caracteres diferente.
    - Se espera a que cada hilo finalice.
    - Se usa la misma función complementaria.
    - Presenta una condición de carrera.
  - El ejemplo 2 utiliza dos hilos en lugar de 3.
  - La utilización de una variable de condición fijará el orden en que deben ejecutarse los hilos, eliminando la condición de carrera.
  - Se quiere mantener el orden en el cual imprimen en pantalla los hilos: primero 'A' y luego 'B'.
  - Se combina el uso de mutex y de variable de condición: se define una estructura para pasar el puntero a cada hilo
    - `shared_state_t`
      - Bandera
      - Mutex
      - Variable de condición

# Variables de condición POSIX

- Ejemplo 2
  - Ambos hilos tienen la posibilidad de ejecutarse primero.
  - Si el hilo 1 bloquea al mutex, imprime su mensaje, setea la bandera y notifica al hilo 2 que podría estar esperando en la variable de condición. La notificación se realiza llamando a la función `pthread_cond_signal()`.
  - Si el hilo 2 bloquea al mutex primero, chequea la bandera y si es `FALSE` significa que el hilo 1 aún no imprimió su mensaje. Así, el hilo 2 espera en la variable de condición y libera la CPU. El mutex antes bloqueado por el hilo 2, se libera al esperar la variable de condición para permitir al hilo 1 poder bloquearlo.

# Barreras POSIX

- Permiten que un grupo de hilos puedan reunirse, reorganizarse y luego continuar desde un punto común.
- Algunos hilos pueden hacer sus tareas más rápido que otros, entonces a veces es necesario tener un punto de encuentro.
- Una barrera se utiliza en casos donde se requiera un control donde los hilos deben detenerse y esperar a que los demás se unan al grupo.
- Funciones
  - `pthread_barrier_init()`: inicializa la barrera asignando un contador de hilos.
  - `pthread_barrier_wait()`: el hilo espera hasta que la cantidad de hilos en espera iguala al contador.

# Barreras

- Ejemplo 3
  - Mediante el uso de barreras se pretende proporcionar una solución similar a la del ejemplo 2 que utiliza mutex/variables de condición para eliminar la condición de carrera.
  - En este ejemplo se quiere que siempre se imprima 'A' y después 'B', independientemente de los entrelazamientos.
  - Se tienen dos funciones complementarias desde donde se llama a la función `pthread_barrier_wait()`.

# Barreras

- Ejemplo 3
  - Generalmente, el uso de barreras lleva a códigos más pequeños que una solución con mutex y variables de condición para sincronizar en algún punto de control.
  - La barrera se declara como global.
  - Se inicializa la barrera con un valor igual a 2, lo que indica la cantidad de hilos que deben quedar en espera llamando a `pthread_barrier_wait()`, y sólo después de eso se liberan todos y pueden continuar con su ejecución.

# Semáforos POSIX

- Controlan el acceso a secciones críticas o recursos compartidos.
- Semáforo general
  - En algunas ocasiones es posible que se desee tener más de un hilo dentro de la sección crítica y operar con el recurso compartido.
- Semáforo binario
  - Sólo un hilo puede ingresar a la sección crítica.
  - Los mutex son semáforos binarios, donde un solo hilo puede acceder al recurso compartido.
- Se debe incluir el header “semaphore.h” ya que los semáforos se agregan como una extensión de la biblioteca pthread.
- Ámbito de un semáforo
  - Semáforos sin nombre
    - Se pueden usar dentro de un proceso y los hilos en ejecución dentro de él.
      - `sem_t local_semaphore;`
      - `semaphore = &local_semaphore;`
      - `sem_init(semaphore, 0, 1);`
  - Semáforo con nombre
    - Se pueden usar en todo el sistema y pueden ser vistos y utilizados por varios procesos en el sistema.
      - `semaphore = sem_open("sem0", O_CREAT | O_EXCL, 0644, 1);`

# Semáforos POSIX

- Comportamiento
  - Cada objeto semáforo tiene un valor entero.
  - Siempre que un hilo espera un semáforo llamando a la función `sem_wait()`, si el valor del semáforo es mayor que 0, entonces el valor se reduce en 1 y el hilo puede ingresar a la sección crítica.
  - Si el valor del semáforo es 0, el hilo debe esperar hasta que el valor del semáforo vuelva a ser positivo.
  - Siempre que un hilo sale de la sección crítica llamando a la función `sem_post()`, el valor del semáforo se incrementa en 1.
  - Por lo tanto, al especificar el valor inicial igual a 1, eventualmente se obtiene un semáforo binario.

# Semáforo binario

- Ejemplo 4
  - Solución de carrera de datos usando semáforos binarios para el ejemplo 5 de la presentación de hilos.
    - Carrera de datos mediante un estado compartido (variable compartida).
    - Los compiten para leer/escribir datos.
    - El ejemplo usa una variable del tipo entera.
    - Las salidas correctas serían:
      - Que se imprima: 1 3
      - Que se imprima: 2 3
    - Una salida incorrecta sería: 1 2 o 2 1

# Semáforo binario

- Ejemplo 4
  - Se declara un puntero al semáforo como global.
  - El semáforo se inicializa con la función `sem_init()` a la cual se le pasa el número máximo de hilos que se permiten en la sección crítica: se pasa el valor 1. El semáforo se comporta como un mutex.
  - En las funciones complementarias, las secciones críticas están protegidas por las funciones `sem_wait()` y `sem_post()`.

# Semáforos generales

- Ejemplo 5
  - Escenario en el que se permite que varios hilos entren en la sección crítica.
  - Ejemplo clásico: implica la creación de 50 moléculas de agua. Para 50 moléculas de agua, se deben tener 50 átomos de oxígeno y 100 átomos de hidrógeno. Si se simula cada átomo usando hilos, se necesitan dos hilos de hidrógeno y un hilo de oxígeno para entrar en sus secciones críticas, con el fin de generar una molécula de agua y contabilizarla.

# Semáforos generales

- Ejemplo 5
  - Primero se crean los 50 hilos de oxígeno y luego los 100 hilos de hidrógeno.
  - Para proteger la sección crítica del hilo de oxígeno, se usa un mutex, pero para las secciones críticas de los hilos de hidrógeno, se usa un semáforo general que permite que dos hilos entren en la sección crítica simultáneamente.
  - Se utilizan barreras para la señalización.

# Semáforos generales

- Ejemplo 5
  - Se declara un mutex como global (`oxygen_mutex`) que protege a las secciones críticas de los hilos de oxígeno. En todo momento, un solo hilo de oxígeno puede ingresar a la sección crítica.
  - El hilo de oxígeno espera, en su sección crítica, que se unan otros dos hilos de hidrógeno para incrementar el contador de moléculas de agua. El incremento ocurre dentro de la sección crítica de oxígeno.
  - Se declara un semáforo general (`hydrogen_sem`) que protege a las secciones críticas de los hilos de hidrógeno. En todo momento, sólo dos hilos de hidrógeno pueden entrar en sus secciones críticas. Allí esperan en la barrera compartida entre los hilos de oxígeno e hidrógeno.
  - Cuando el número de hilos en espera en el objeto de barrera compartido llega a tres, significa que se tiene un oxígeno y dos hidrógenos: se crea una molécula de agua y todos los hilos en espera pueden continuar. Los hilos de hidrógeno salen inmediatamente, y el hilo de oxígeno incrementa el contador de moléculas de agua para luego salir.
  - El proceso espera a que todos los hilos finalicen usando `pthread_join()`.



Centro Regional Universitario Córdoba IUA  
**Ing. en Informática**

**¿Preguntas?**